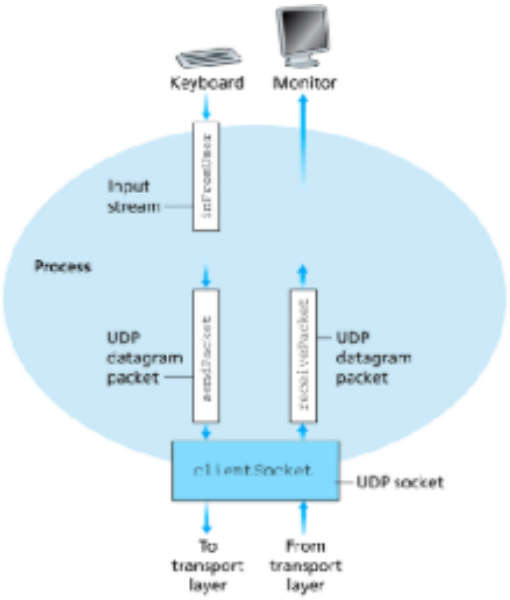
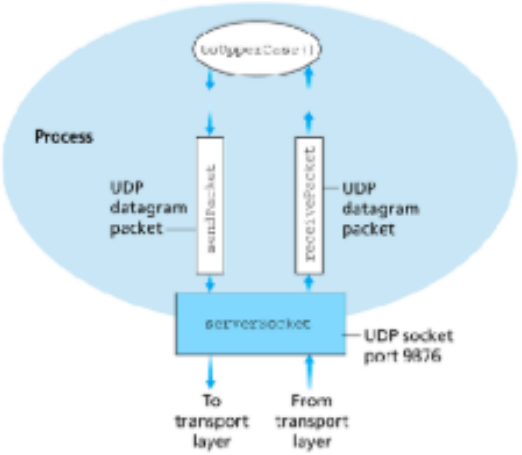
Federico D'Amato 5°Binf V1.2 **11/12/2025 - 17/12/2025**

Comunicazione tramite socket UDP

Esercizio B - Messaggio in maiuscolo **2**
Descrizione del Problema 2
Progettazione 2
Schema Logico 2
Implementazione 2
Collaudo 2
Procedimento e Risultati 3
Considerazioni Finali 4
Esercizio C - UDP "DayTime" con conteggio connessioni **5**
Descrizione del Problema 5
Progettazione 5
Schema Logico 5
Implementazione 6
Collaudo 6
Procedimento e Risultati 7
Considerazioni Finali 8





Esercizio B - Messaggio in maiuscolo

Descrizione del Problema

Realizzare un'applicazione client-server in Java che utilizzi il protocollo UDP, in cui il client invia un messaggio testuale al server. Il server riceve il messaggio, lo converte in maiuscolo e lo rinvia al client, che visualizza la risposta a schermo.

Progettazione

L'applicazione è strutturata secondo il modello client-server utilizzando il protocollo UDP. Il client (Sender) invia una stringa al server (Receiver), il quale riceve il messaggio, lo converte in maiuscolo e lo rinvia al client.
La comunicazione avviene in locale tramite l'indirizzo IP 127.0.0.1 e una porta predefinita.

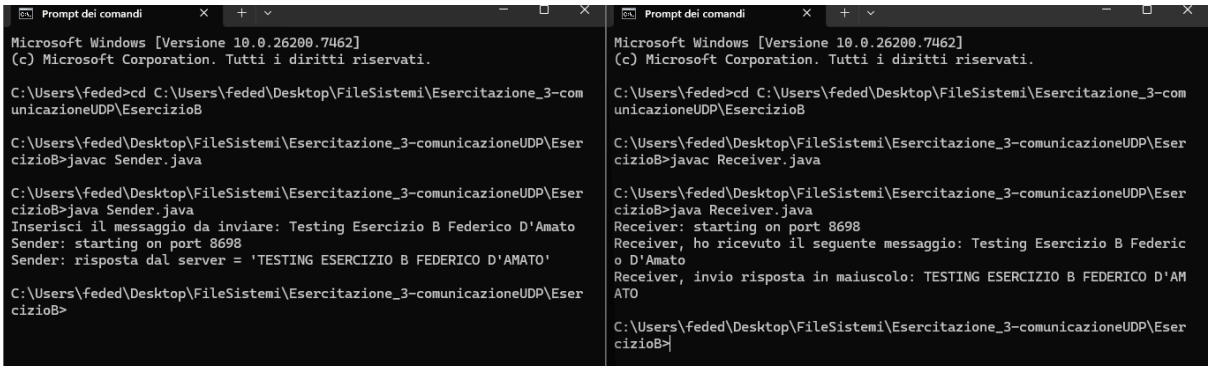
Schema Logico

- Il Sender legge un messaggio inserito dall'utente
- Il Sender invia il messaggio al Receiver tramite un datagramma UDP
- Il Receiver riceve il messaggio
- Il Receiver converte il testo in maiuscolo
- Il Receiver invia la risposta al Sender
- Il Sender riceve il messaggio trasformato e lo visualizza

Implementazione

- Sender
- Receiver

Collaudo



```
Prompt dei comandi
Microsoft Windows [Versione 10.0.26200.7462]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\feded>cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-com
unicazioneUDP\EsercizioB

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>javac Sender.java

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>java Sender.java
Inserisci il messaggio da inviare: Testing Esercizio B Federico D'Amato
Sender: starting on port 8698
Sender: risposta dal server = 'TESTING ESERCIZIO B FEDERICO D'AMATO'

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>
```

```
Prompt dei comandi
Microsoft Windows [Versione 10.0.26200.7462]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\feded>cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-com
unicazioneUDP\EsercizioB

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>javac Receiver.java

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>java Receiver.java
Receiver: starting on port 8698
Receiver, ho ricevuto il seguente messaggio: Testing Esercizio B Federic
o D'Amato
Receiver, invio risposta in maiuscolo: TESTING ESERCIZIO B FEDERICO D'AM
ATO

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>
```

```
Prompt dei comandi

Microsoft Windows [Versione 10.0.26200.7462]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\feded>cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-com
unicazioneUDP\EsercizioB

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>javac Receiver.java

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>java Receiver.java
Receiver: starting on port 8698
Receiver, ho ricevuto il seguente messaggio: Testing Esercizio B Federic
o D'Amato
Receiver, invio risposta in maiuscolo: TESTING ESERCIZIO B FEDERICO D'AM
ATO

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>
```

```
Prompt dei comandi

Microsoft Windows [Versione 10.0.26200.7462]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\feded>cd C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-com
unicazioneUDP\EsercizioB

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>javac Sender.java

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>java Sender.java
Inserisci il messaggio da inviare: Testing Esercizio B Federico D'Amato
Sender: starting on port 8698
Sender: risposta dal server = 'TESTING ESERCIZIO B FEDERICO D'AMATO'

C:\Users\feded\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\Eser
cizioB>
```

Procedimento e Risultati

Per realizzare l'applicazione è stato seguito un approccio graduale. In primo luogo è stato sviluppato il server (Receiver), che si mette in ascolto su una porta predefinita utilizzando un DatagramSocket. Una volta ricevuto il messaggio dal client, il server lo converte in maiuscolo tramite il metodo toUpperCase() e invia la risposta al mittente utilizzando le informazioni di indirizzo e porta contenute nel pacchetto ricevuto.

Successivamente è stato implementato il client (Sender), che legge il messaggio inserito dall'utente da tastiera, crea un pacchetto UDP e lo invia al server specificando indirizzo IP e porta. Dopo l'invio, il client resta in attesa della risposta del server e, una volta ricevuta, la stampa a schermo.
Durante le prove di esecuzione, avviando prima il Receiver e poi il Sender, il sistema ha funzionato correttamente: il messaggio inserito dal client è stato ricevuto dal server, trasformato in maiuscolo e restituito senza errori. I risultati ottenuti confermano il corretto funzionamento della comunicazione client-server tramite protocollo UDP e il raggiungimento dell'obiettivo dell'assegnato.

Considerazioni Finali

L'esercizio ha permesso di comprendere il funzionamento della comunicazione UDP in Java e il modello client-server. Il protocollo UDP risulta semplice e veloce, ma non garantisce l'affidabilità della trasmissione. Per applicazioni più complesse o critiche sarebbe preferibile utilizzare il protocollo TCP. L'obiettivo dell'esercizio è stato pienamente raggiunto, dimostrando una corretta gestione dei socket e dello scambio di messaggi.

Esercizio C - UDP “DayTime” con conteggio connessioni

Descrizione del Problema

Si prevede la realizzazione di un server UDP in ascolto sulla porta 1313, che risponde a ogni pacchetto ricevuto inviando la data e l’ora correnti in formato leggibile. Il server deve tenere traccia del numero di connessioni di ciascun client, identificato tramite indirizzo IP, salvando i dati su file in modo persistente.
Per quanto riguarda le prime nove connessioni il servizio è gratuito, alla decima viene segnalato l’esaurimento dei bonus e, di conseguenza, dalle connessioni successive il server risponde solo con un messaggio che indica il passaggio ad un servizio a pagamento, per il quale è richiesta una API key.

Progettazione

L’applicazione è basata su un’architettura client–server che utilizza il protocollo UDP. Il server è in ascolto sulla porta 1313 e, ogni volta che riceve un pacchetto da un client, elabora la richiesta e invia una risposta. Normalmente la risposta consiste nella data e nell’ora correnti in formato leggibile.

Il server tiene traccia del numero di connessioni effettuate da ogni client, identificandolo tramite il suo indirizzo IP. Queste informazioni vengono salvate in un file, in modo che i dati rimangano disponibili anche dopo il riavvio del server.

Per le prime nove richieste di ciascun client il servizio è gratuito. Alla decima richiesta il server avvisa che i bonus sono esauriti; dalle richieste successive in poi, il server non fornisce più la data e l’ora, ma invia un messaggio che indica il passaggio a un servizio a pagamento, per il quale è richiesta una API key.

Schema Logico

- Il Client invia un pacchetto UDP al Server
- Il Server riceve il pacchetto
- Il Server identifica il client tramite il suo indirizzo IP
- Il Server legge dal file il numero di connessioni associate a quell’IP
- Il Server incrementa il contatore delle connessioni
- Il Server salva il nuovo valore nel file
- Il Server verifica quante richieste ha fatto il client:
    - fino a 9 → servizio gratuito
    - alla 10ª → avviso di fine bonus
    - oltre la 10ª → richiesta di API key
- In base al risultato, il Server prepara la risposta:
    - data e ora correnti, oppure

- messaggio di servizio a pagamento
- Il Server invia la risposta al Client
- Il Client riceve il messaggio e lo visualizza

Implementazione

- Sender
- Receiver

Collaudo

Client

```
Prompt dei comandi - java Se
Microsoft Windows [Versione 10.0.26200.8039]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\Federico>cd C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>javac Sender.java

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java

Risposta: 2026-03-25 23:04:57

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:04:59

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:01

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:03

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:05

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:07

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:10

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:12

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: 2026-03-25 23:05:15

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: Bonus esauriti! Il servizio è ora a pagamento.

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: Inserire la API KEY per sbloccare il servizio.
Inserisci API KEY: 1
Risposta con API KEY: Inserire la API KEY per sbloccare il servizio.

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Sender.java
Risposta: Inserire la API KEY per sbloccare il servizio.
Inserisci API KEY: 12345
Risposta con API KEY: Accesso consentito con API KEY: 2026-03-25 23:05:50
```

Server

```
Prompt dei comandi - java R

Microsoft Windows [Versione 10.0.26200.8039]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\Federico>cd C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>javac Receiver.java

C:\Users\Federico\Desktop\FileSistemi\Esercitazione_3-comunicazioneUDP\EsercizioC>java Receiver.java

Server UDP avviato sulla porta 1313
IP: 127.0.0.1 | Connessione: 1 | Richiesta: richiesta | Risposta: 2026-03-25 23:04:57
IP: 127.0.0.1 | Connessione: 2 | Richiesta: richiesta | Risposta: 2026-03-25 23:04:59
IP: 127.0.0.1 | Connessione: 3 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:01
IP: 127.0.0.1 | Connessione: 4 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:03
IP: 127.0.0.1 | Connessione: 5 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:05
IP: 127.0.0.1 | Connessione: 6 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:07
IP: 127.0.0.1 | Connessione: 7 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:10
IP: 127.0.0.1 | Connessione: 8 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:12
IP: 127.0.0.1 | Connessione: 9 | Richiesta: richiesta | Risposta: 2026-03-25 23:05:15
IP: 127.0.0.1 | Connessione: 10 | Richiesta: richiesta | Risposta: Bonus esauriti! Il servizio è ora a
 pagamento.
IP: 127.0.0.1 | Connessione: 11 | Richiesta: richiesta | Risposta: Inserire la API KEY per sbloccare i
l servizio.
IP: 127.0.0.1 | Connessione: 12 | Richiesta: 1 | Risposta: Inserire la API KEY per sbloccare il serviz
io.
IP: 127.0.0.1 | Connessione: 13 | Richiesta: richiesta | Risposta: Inserire la API KEY per sbloccare i
l servizio.
IP: 127.0.0.1 | Connessione: 14 | Richiesta: 12345 | Risposta: Accesso consentito con API KEY: 2026-03
-25 23:05:50
IP: 127.0.0.1 | Connessione: 15 | Richiesta: richiesta | Risposta: Inserire la API KEY per sbloccare i
l servizio.
```

Procedimento e Risultati

Per realizzare l'applicazione è stato seguito un approccio graduale. In primo luogo è stato sviluppato il server (Receiver), che all'avvio carica da un file di testo (connections.txt) gli indirizzi IP già noti e il numero di connessioni effettuate da ciascuno. Il server si mette in ascolto sulla porta 1313 tramite un DatagramSocket. Ogni volta che arriva un pacchetto, estrae l'IP del mittente, incrementa il relativo contatore e aggiorna il file. A questo punto, in base al conteggio, determina la risposta: fino a 9 connessioni restituisce la data e ora corrente, alla decima comunica che i bonus gratuiti sono terminati e dalle successive richiede una chiave API, concedendo nuovamente l'accesso al suo corretto inserimento.
Successivamente è stato implementato il client (Sender), che invia una prima richiesta al server e attende la risposta. Se questa contiene la frase "API KEY", il programma chiede all'utente di digitare la chiave, la invia e stampa la nuova risposta.
Durante le prove, avviando prima il server e poi i client, il sistema ha funzionato correttamente. Ogni IP ha mantenuto un contatore indipendente: i primi nove messaggi hanno ricevuto la data e ora, il decimo ha segnalato l'esaurimento dei bonus, e con la chiave corretta è stato nuovamente concesso l'accesso. Il file connections.txt è stato aggiornato coerentemente, dimostrando la corretta persistenza dei dati.

Considerazioni Finali

L’esercizio ha permesso di approfondire la comunicazione UDP in Java e il modello client-server, arricchendo l’implementazione con persistenza su file e logica basata sul numero di accessi. Il protocollo UDP si è rivelato adatto per la sua semplicità, ma non garantisce affidabilità né ordine dei pacchetti. In un’applicazione reale sarebbero necessari accorgimenti per la concorrenza e la sicurezza (ad esempio cifratura della chiave API). Nonostante ciò, l’obiettivo principale è stato pienamente raggiunto: realizzare un’applicazione funzionante con conteggio per singolo client e un successivo controllo tramite chiave.